

Ch01 軟體危機、品質模型與 AI 時代的可靠性工程

Chapter 01: The Software Crisis, Quality Models, and AI-Era Reliability Engineering

😊 大家都知道「物質不滅定律」；身為資工系學生，我們更熟悉「Bug 不滅定律」。

在 2026 年，寫出一段程式碼只要問 AI 3 秒鐘；但要證明這段程式碼在生產環境不會搞垮公司，可能要花上 3 個月。

📌 本章目錄與重點導讀 (Table of Contents & Highlights)

軟體工程的核心使命，是為人類社會打造可靠、安全且合用的數位基礎設施。本章作為 SQA 課程的總覽與基石，將從歷史浩劫出發，穿透 AI 時代的工程迷思，建立起現代軟體品質的立體思維模型：

Ch01 知識架構全景：

- 【歷史與現況】1.1 軟體危機四大歷史慘劇 → 1.2 AI 時代的新軟體危機與品質事件
- 【品質的本質】1.3 軟體四要素 (IEEE) & Garvin 五大品質觀點
- 【工程與經濟】1.4 驗證與確認 (V&V) & 品質成本 (CoQ 1:10:100 定律)
- 【流程與門檻】1.5 SDLC 生命週期、V 模型 & DevOps 6 大連續品質門檻
- 【國際標準規約】1.6 ISO 25010 八大產品品質特性 & 情境挑戰賽

章節單元	核心學習重點 (Key Takeaways)
1.1 軟體危機的歷史與 AI 時代的輪迴	透過愛國者飛彈、火星探測器、華航名古屋空難、迪士尼獅子王四大歷史慘劇，理解精度誤差、介面契約、人機互動與相容性測試的重要性，剖析 1968 NATO 軟體危機的本質。
1.2 AI 能拯救軟體危機嗎？	揭露 AI 輔助開發的「虛假安全感」與技術債；剖析幻覺套件投毒 (Slopsquatting)、亞馬遜大斷線、Vibe Coding 漏洞與金鑰外洩等真實事故，確立「從寫程式碼轉向驗證程式碼」的思維轉型。
1.3 軟體的本質與品質維度	掌握 IEEE 610.12 軟體四大組成要素（程式、程序、文件、資料）；深入解析 David Garvin 五大品質觀點（超自然、使用者、製造、產品、價值觀點）。
1.4 軟體品質工程核心概念	辨析 Verification（是否有正確建造軟體）vs. Validation（建造的是否是正確軟體）；理解品質成本架構 (CoQ) 與 1:10:100 缺陷修復倍增定律，奠定「測試左移 (Shift-Left)」的經濟學基礎。
1.5 生命週期中的品質把關	探索 V 模型的測試與開發對稱性，解析現代 DevOps CI/CD 流水線中的 6 大連續品質門檻 (Quality Gates) (Pre-commit → SAST → Unit → Integration → E2E → Observability)。
1.6 現代軟體品質模型 ISO 25010	掌握國際標準 ISO 25010 八大產品品質特性，探索 ISO 25023 量化指標 (Metrics) 與工程落地，並透過 10 題情境連環戰實戰辨析。
1.7 綜合練習與思維激盪	結合理論與實務，引導進行 AI 時代品質反思、ISO 25010 案例分析與數值精度累計實作。

1.1 軟體危機的歷史與 AI 時代的輪迴

軟體既能造福人類，亦能造成毀滅性災難。回顧歷史，軟體缺陷曾引發嚴重的空難、軍事傷亡與數億美元的太空浩劫。

1.1.1 Case 1：愛國者反導彈事件 (1991) —— 毫秒級的精度累積誤差

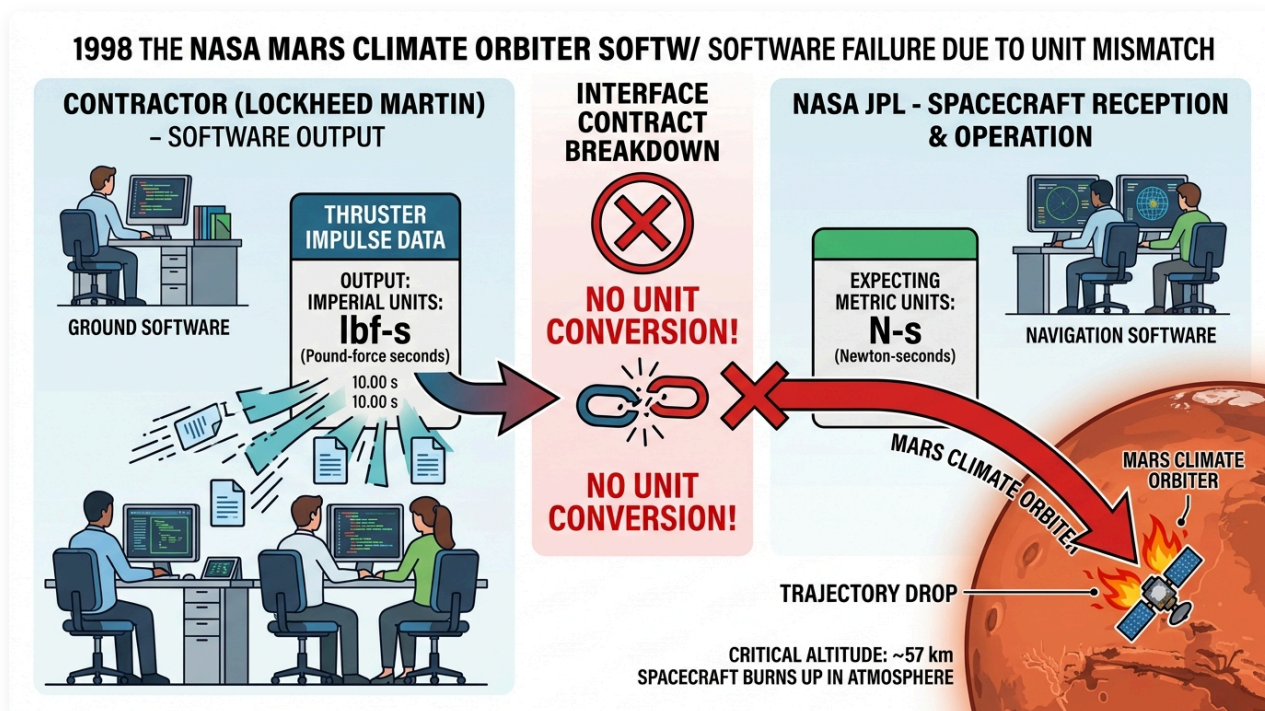
在 1991 年 2 月波斯灣戰爭中，一枚伊拉克發射的飛毛腿飛彈擊中美軍沙烏地達蘭基地，造成 28 名美軍死亡、100 多人受傷。

- 致命軟體缺陷：愛國者系統時鐘暫存器採用 24-bit 浮點數 設計，將時間轉換為 0.1 秒單位時產生了微小的截斷誤差（約 0.000000095 秒）。
- 災難放大：雷達系統連續開機運作超過 100 小時 未重啟，微小的精度誤差累計達 0.33 秒。

- **致命後果：**飛毛腿飛彈速度達 4.2 馬赫 (1.5 km/s)，0.33 秒相當於 600 公尺距離偏差。雷達搜尋窗無法鎖定目標，攔截飛彈根本沒有發射。
- **SQA 啟示：**數值精度問題、浮點數累計誤差，以及**長時運行可靠度測試 (Long-term Stress/Reliability Testing) **的重要性。

1.1.2 Case 2：NASA 火星氣候軌道探測器 (1998) —— 單位的代價

1998 年 NASA 發射「火星氣候軌道探測器」(Mars Climate Orbiter，造價近 2 億美元)，抵達火星後失聯焚毀。

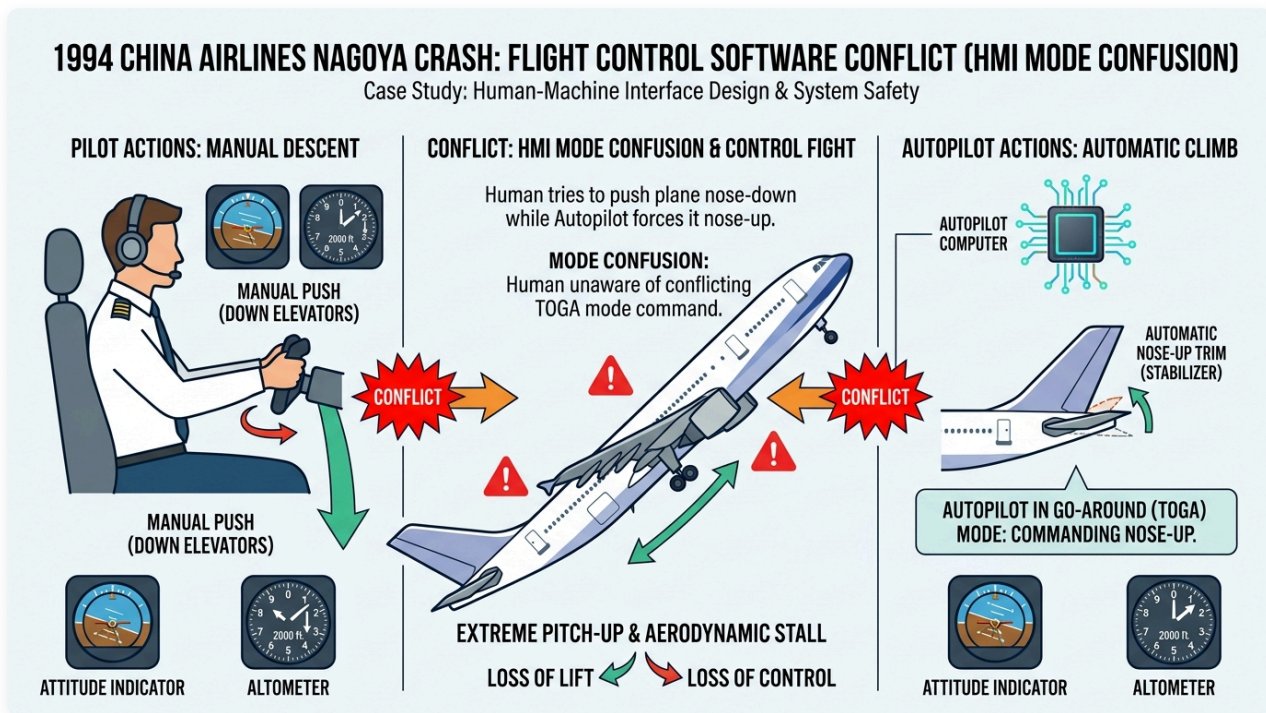


圖形解說：跨模組介面契約斷裂導致太空船墜毀

- **【左側】** 承包商軟體端 (洛克希德馬丁)：地面控制程式以 英制單位 (磅力·秒，lbf·s) 輸出推進器衝量數據。
- **【中間】** 介面契約斷裂 (Interface Contract Breakdown)：兩端系統缺乏嚴謹的介面型態定義與自動化單位轉換機制。
- **【右側】** NASA JPL 導航接收端：太空船導航軟體預設以 公制單位 (牛頓·秒，N·s) 解析輸入數據，導致推力計算出現 4.45 倍的嚴重偏差。
- **災難後果：**軌道高度預計 140 公里，實際暴跌至 57 公里，直接在火星大氣層中摩擦燃燒解體。
- **SQA 啟示：**跨模組介面契約 (Interface Contract)、強型態檢驗與規格審查的重要性。

1.1.3 Case 3：華航名古屋空難 (1994) —— 人機爭奪控制權

1994 年 4 月 26 日，華航 CI140 班機（空中巴士 A300-622R）在名古屋機場降落時墜毀，264 人罹難。



圖形解說：人機介面衝突 (HMI Mode Confusion) 與控制權仲裁缺失

- **【左側】機師手動操作 (Manual Push)：**副駕駛誤觸「重飛 (Go-Around / TOGA)」模式後，正副駕駛試圖手動前推操縱桿 (Down Elevators) 強壓機首下降以利降落。
- **【右側】飛控電腦自動配平 (Autopilot Automatic Climb)：**機載飛控電腦因處於「自動重飛」模式，強行將水平安定面 (Horizontal Stabilizer) 向上配平以抬高機首爬升。
- **【中央衝突】模式混淆與控制權爭奪 (Control Fight)：**駕駛員未察覺電腦仍在執行重飛指令，人機力量相互抵消；最終水平安定面達到極限仰角，飛機在低空發生氣動失速 (Aerodynamic Stall) 墜毀。
- **SQA 啟示：**人機互動 (HMI/UX) 狀態透明度、異常操作回饋與自動化控制權限仲裁設計。

1.1.4 Case 4：迪士尼《獅子王》遊戲 (1994) —— 缺乏相容性測試的公關浩劫

1994 年聖誕節迪士尼推出《獅子王》PC 遊戲，伴隨 Compaq 等家用電腦熱銷，數以萬計家庭期待同樂。

- **致命缺陷：**遊戲基於特定視訊驅動 (WinG) 開發，未在市場主流多樣硬體環境上進行充分相容性測試。
- **災難後果：**大量家用電腦開機即藍屏崩潰，聖誕節當天客服被憤怒家長打爆，嚴重損害品牌聲譽。
- **SQA 啟示：**環境多樣性驗證與相容性測試 (Compatibility Testing)，促使微軟後來開發標準化 DirectX 架構。

1.1.5 軟體危機的定義與成因

1968 年 NATO 會議首次提出「軟體危機 (Software Crisis)」：硬體飛速發展，而軟體開發卻面臨預算超支、進度延期、品質低下、維護困難等問題。軟體危機的原因可以歸納為以下幾點：

1. 軟體複雜性：隨著電腦硬體性能的提升，軟體的規模和複雜性不斷增加，超出了傳統軟體開發方法的處理能力。
2. 軟體開發效率低下：軟體開發的進度和成本往往難以預測，開發團隊規模不斷擴大，開發效率卻沒有同步提升。
3. 軟體品質低下：軟體錯誤率高，軟體品質難以保證，導致軟體系統不穩定，甚至引發嚴重的安全事故。
4. 軟體維護困難：隨著軟體規模的擴大，軟體維護的難度不斷增加，軟體開發團隊難以適應軟體維護的需求。

概念核對問答 (CCQ 1)

問題

愛國者反導彈系統（1991）在達蘭基地攔截失效的根本軟體原因為何？

- A) 通訊網路中斷導致雷達無法傳送指令給飛彈發射架
- B) 24-bit 時鐘暫存器的浮點捨入誤差在連續運行 100 小時後累加達 0.33 秒
- C) 程式碼發生記憶體洩漏 (Memory Leak) 導致作業系統當機
- D) 雷達演算法誤將美軍戰機辨識為敵方飛毛腿飛彈

課堂互動

雙人課堂討論 (Pair Discussion) —— 真實世界的軟體失敗案例

- **討論任務：**請與鄰近同學組成雙人小組，分享一件你曾遇過、聽過，或透過網路搜尋找到的真實軟體失敗/事故案例（例如：2024 年 CrowdStrike 全球藍屏事件、Knight Capital 交易系統 45 分鐘虧損 4.6 億美元、熱門售票系統或遊戲上線當機等）。
- **引導思考與討論：**
 - i. **事件情境與影響：**該系統發生了什麼異常？對使用者、企業營運或整體社會帶來了哪些具體的衝擊與損失？
 - ii. **根本原因 (Root Cause)：**為什麼會發生這個錯誤？（是需求誤解、邏輯缺陷、數值捨入誤差、並行競爭、缺乏程式碼審查，還是部署流程漏洞？）
 - iii. **預防策略 (Prevention)：**若站在軟體品質保證 (SQA) 與軟體測試的角度，團隊應採取哪些防護機制或工程實踐（例如：單元測試、自動化回歸測試、靜態分析、金絲雀發布、容錯設計等）來避免類似問題發生？

課堂互動

1.2 AI 能拯救軟體危機嗎？

近年來，隨著人工智慧（AI）技術的突飛猛進，AI 輔助開發工具（如 GitHub Copilot、ChatGPT）的普及，似乎為軟體工程界注入了一劑強心針。然而，數據顯示這並非萬靈丹，反而可能加劇了隱藏的危機。近年的多項研究與學術調查提供了驚人的數據支持：

1. 程式碼維護性惡化：GitClear 縱向研究 (2020–2026)

- GitClear 分析了 1.5 億行程式碼的 Git Commit 數據，發現自 AI 輔助工具普及以來：
 - 程式碼重複率 (Code Duplication) 呈指數級上升。
 - 衡量程式碼重構的關鍵指標「移動行數 (Moved Lines)」大幅下降，表示工程師更少主動去重構、整理舊程式碼。
 - 程式碼流失率 (Code Churn) 顯著增高（剛寫好的程式碼在短時間內被刪除或重寫），這證明 AI 生成了大量看似可行、實則脆弱的程式碼，帶來了沈重的長期維護性債務 (Maintainability Debt) [5]。

2. 52% 的高錯誤率與「虛假安全感」：Purdue University 實證研究

- 普渡大學研究團隊評估了 ChatGPT 在回答 Stack Overflow 上的 517 個軟體工程問題時的表現：
 - 結果顯示，AI 的解答中有 52% 包含錯誤的程式碼或資訊。
 - 但更可怕的是，由於 AI 的語氣極度禮貌、條理清晰且「看似極度合理」，有高達 39.3% 的使用者依然偏好並採信了 AI 的錯誤回答。
 - 這會使工程師產生錯誤的「安全感」，未經嚴謹驗證就直接將漏洞帶入系統中 [6]。

3. 40% 的安全弱點隱患：紐約大學 (NYU) 等學術研究

- 研究人員對 AI 生成的程式碼進行自動化安全掃描（基於 Common Weakness Enumeration, CWE 標準）：結果發現，AI 在沒有特定安全提示的引導下，生成程式碼中有高達約 40% 包含已知的安全弱點（如：緩衝區溢位、SQL 注入、並發競爭危害）[7]。

1.2.1 AI 寫程式引發的品質事件

1. 幻覺套件供應鏈投毒 (Slopsquatting / Package Hallucination)

- 事故機制：LLM 在撰寫程式碼時，常會「一本正經地捏造」一個聽起來非常合理的套件名稱（例如 `crypto-validator` 或 `huggingface-cli`）。
- 釀成災難：黑客監控 AI 常出現的幻覺套件名稱後，搶先在 PyPI 或 npm 上註冊同名惡意套件。不知情的工程師直接執行 AI 給的 `pip install` 或 `npm install` 指令，導致後門程式與木馬直接植入企業開發環境與生產伺服器。研究顯示有惡意概念驗證套件在數月內被無辜開發者下載超過數萬次 [1]。

2. 亞馬遜 (Amazon) 電商系統大斷線與訂單蒸發

- 事故機制：2026 年 3 月上旬，亞馬遜電商系統遭遇嚴重故障，內部文件一度指出工程師使用 AI 寫程式工具輔助生成變更程式碼，但未經完整審查與自動化防護驗證即推上生產環境。
- 釀成災難：導致送貨與結帳邏輯出錯，引發北美訂單一度崩跌 99%，在數小時內蒸發超過 630 萬筆訂單與大量營收 [2]。

3. Vibe Coding 帶來的漏洞大爆發（以 Lovable/No-code 平台為例）

- **事故機制：**非工程背景人員或初階開發者僅憑 Prompt 快速產出整套 Web 服務，缺乏程式碼審查（Code Review）與安全防護概念。
- **釀成災難：**資安團隊針對 AI 自動生成的 1,600 多個上線應用進行掃描，發現高達 10% 以上存在嚴重的 SQL Injection、越權存取（Broken Access Control）或敏感資料外洩漏洞，用戶可輕易繞過驗證直接進入管理後台 [3]。

4. 敏感金鑰與憑證直接寫死（Hardcoded Secrets）外洩

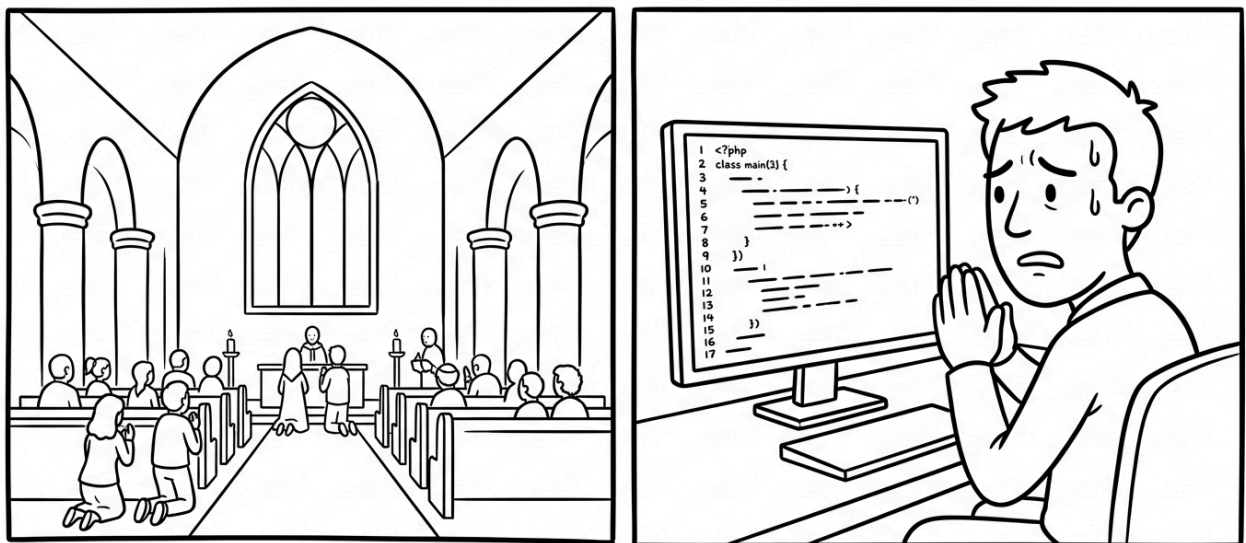
- **事故機制：**AI 為了讓程式「立刻能跑」，經常在範例程式碼中示範把 API Key、資料庫密碼寫死在程式碼內。
- **釀成災難：**開發者在沒有抽換成環境變數（Environment Variables）的情況下，直接將程式碼推送到公開 GitHub Repository，導致雲端帳號（如 AWS、OpenAI）在一小時內被掃描機器人盜用並產生數萬美元的巨額帳單 [4]。

💡 結論：

在 2026 AI 時代，軟體開發的真正瓶頸（Bottleneck）已經從「程式碼寫不寫得出來（Writing）」完全轉移到「程式碼到底正不正確（Verification）」。如果開發者缺乏品質保證知識，只盲信 AI 的「綠燈完成」，將會使軟體系統迅速崩潰。

😂 軟體和教堂非常相似——建成之後我們就開始祈禱。

‘Software and cathedrals are much the same – first we build them, then we pray.’ — Sam Redwine



🧠 概念核對問答 (CCQ 2)

問題

在評估生成式 AI（如 GitHub Copilot、ChatGPT）對軟體專案品質的影響時，軟體工程度量研究（如 GitClear）常使用「程式碼流失率（Code Churn）」作為關鍵指標。關於 Code Churn 的定義及其在 AI 時代所反映的品質現象，下列敘述何者最為精準？

- A) 指專案從一個程式語言遷移至另一個語言時，因語法不相容而遺失的程式碼行數比例
- B) 指新寫入並 Commit 的程式碼在極短時間內（如兩週內）就被刪除、修改或替換的比例；高 Code Churn 反映出 AI 生成程式碼看似快速但本質脆弱、未經深思熟慮與充分驗證
- C) 指編譯器與建置工具在優化打包過程中，自動剔除未引用死碼（Dead Code）的效率
- D) 指自動化測試案例因系統版本迭代而自然失效無法執行的比率

參考資料出處 (References)：

1. **Lasso Security: AI Package Hallucinations** — 研究指出 AI 幻覺套件（如 `huggingface-cli`）可能引發 Slopsquatting 攻擊，惡意套件在數月內被無辜下載超過 3 萬次。
2. **CRN: AWS Outage Was Not AI-Caused Via Kiro Coding Tool, Amazon Confirms** — 報導亞馬遜內部大推 AI 寫程式工具 Kiro 以及相關系統故障引發的程式碼安全重整爭議與澄清。
3. **Threat Landscape: Lovable.dev Data Breach: BOLA Vulnerability in Vibe Coding** — 詳細分析 AI 自動建置應用平台 Lovable 於 2026 年爆發的 BOLA (IDOR) 越權漏洞與產生的程式碼/金鑰暴露風險。
4. **GitGuardian: State of Secrets Sprawl Report 2026** — 數據顯示 AI 輔助開發的金鑰與憑證洩漏率是人類開發者的兩倍（如 Claude Code 輔助提交的洩漏率達 3.2%）。
5. **GitClear: Coding on Copilot: 2024 Developer Research** — 針對 1.5 億行程式碼進行的縱向分析，指出 AI 輔助開發使程式碼重複率與流失率增加，並降低了主動重構的頻率。
6. **Purdue University: Is Stack Overflow Obsolete? An Empirical Study of the Characteristics of ChatGPT Answers to Stack Overflow Questions** — 實證研究發現 ChatGPT 在回答軟體工程問題時，52% 的解答包含錯誤程式碼或資訊，且有 39% 的使用者採信了錯誤回答。
7. **New York University (NYU): Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions** — 學術安全掃描研究指出，在無安全提示引導下，AI 生成的程式碼中有約 40% 包含常見的安全弱點（CWE Top 25 漏洞）。

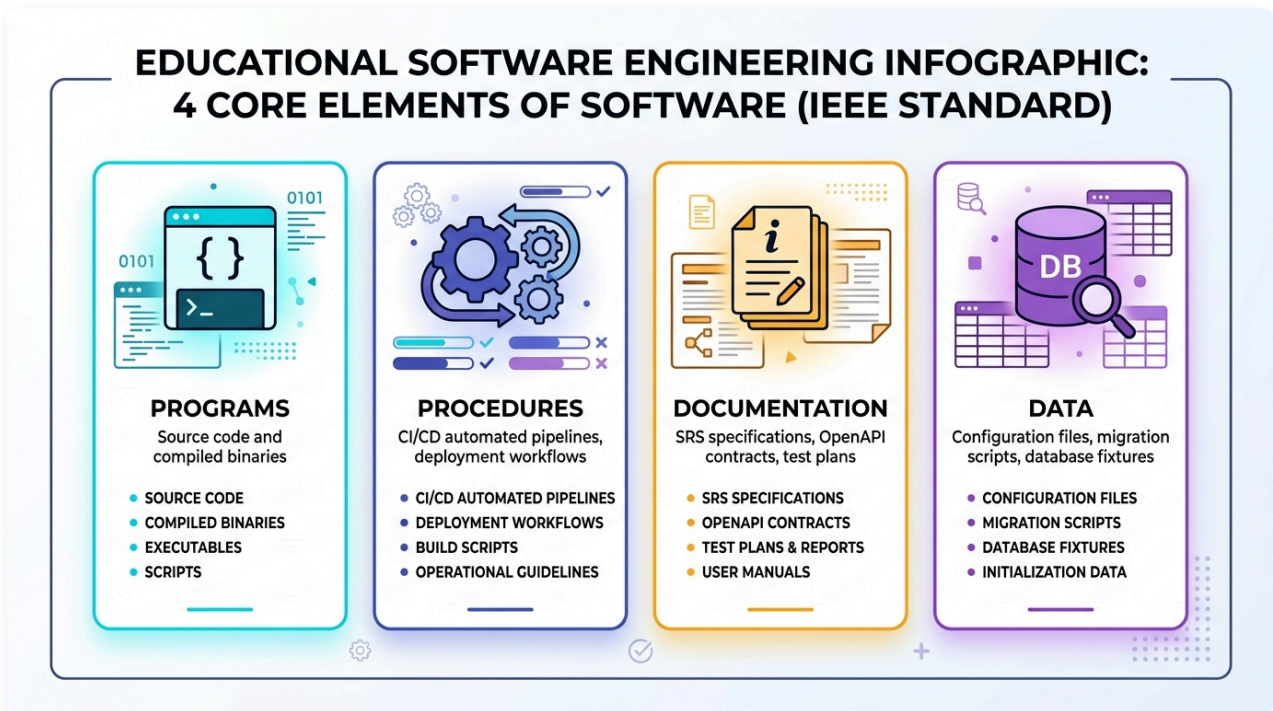
課堂互動

1.3 軟體的本質與品質維度（軟體四要素 & Garvin 五大品質觀點）

1.3.1 軟體的四大組成要素 (IEEE 610.12)

軟體到底是什麼？許多初學者常誤以為「寫出來能跑的原始程式碼」就是軟體。但根據 IEEE (Standard 610.12) 的權威定義，軟體是一個完整的系統化工程有機體：

Software (軟體): Computer programs (程式), procedures (程序), and possibly associated documentation (文件) and data (資料) pertaining to the operation of a computer system.



圖形解說：軟體四大核心要素 (IEEE 610.12)

如果把現代軟體系統（如外送平台、智慧醫療或行動銀行）比喻為一部高速運轉的高鐵列車，這四大要素各自扮演不可或缺的關鍵角色：

1. Programs (程式 / 原始程式碼) —— 「引擎動力與神經網路」：

- **內涵**：包含開發者撰寫的原始碼 (Source Code)、編譯產生的 Bytecode/二進位執行檔、演算法函式庫與微服務 API，負責承載核心業務邏輯。
- **實例**：例如外送平台中計算「外送員最佳派單路徑」與「尖峰時段動態加價」的 Java / Python 核心演算法。
- **SQA 啟示**：光有程式碼就像只有引擎卻沒有軌道、保養手冊與汽油的幽靈車，根本無法在生產環境安全交付。

2. Procedures (作業程序與維運規程) —— 「標準作業流程 SOP 與發布軌道」：

- **內涵**：包含 CI/CD 自動化建置流水線、灰度/金絲雀發布規程 (Canary Deployment)、災難復原演練 (Disaster Recovery)、資料庫定期備份排程與維運手冊 (Runbooks)。

- **實例**：2024 年 7 月 **CrowdStrike 全球大當機**導致全球 850 萬台 Windows 藍屏癱瘓、數千架航班停飛。事故根本原因正是發布程序出現致命漏洞——更新檔未經過分階段逐步發布驗證，一次性推送至全球所有主機。軟體品質保證絕不只檢驗程式碼，更取決於這套維運程序是否具備安全防護！

3. Documentation (文件、規格與契約) —— 「設計藍圖與通訊法典」：

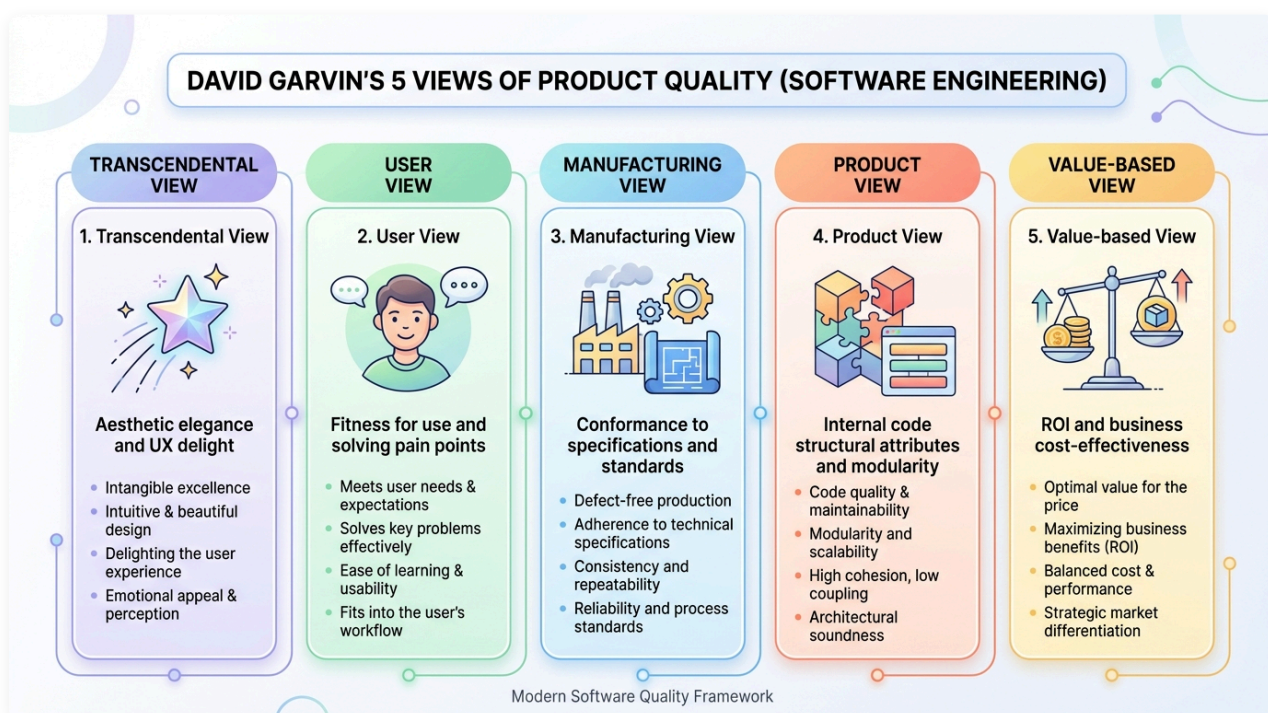
- **內涵**：包含需求規格書 (SRS)、OpenAPI 介面契約、架構設計圖、驗收準則 (Acceptance Criteria) 與使用者操作手冊。在現代工程中，文件更是自動化測試的基石（如 Cucumber Gherkin 規格即活文件）。
- **實例**：1998 年 **NASA 火星氣候軌道探測器**焚毀事故，洛克希德馬丁地面端輸出「英制單位（磅力·秒）」，NASA 導航軟體預設接收「公制單位（牛頓·秒）」。兩邊的程式碼各自都沒有語法錯誤，但因介面文件與通訊契約嚴重斷裂且未嚴格審查，直接燒掉兩億美元！

4. Data (資料、設定檔與測試基準) —— 「血液、燃料與環境配置」：

- **內涵**：包含資料庫初始化結構與遷移腳本 (Schema Migration)、微服務設定檔（`application.prod.yml`）、環境變數與測試測資集 (Test Fixtures / Mock Data)。
- **實例**：程式碼完全沒變，但維運人員在部署時將 `DATABASE_TIMEOUT` 誤設為 `30ms`（原為 `30s`），或者資料庫連線字串少了一個字元，整座系統在上線瞬間引發連鎖雪崩！現代工程倡導「組態即程式碼 (Configuration as Code)」，資料與配置檔案的驗證同樣是測試工程的核心範疇。

1.3.2 何謂品質？David Garvin 的五大品質觀點

當我們探討「軟體品質」時，哈佛商學院教授 David Garvin 在《Managing Quality》中指出，品質並非單一維度，而是由多重視角交織而成的立體概念：



圖形解說：David Garvin 五大品質觀點

1. **超自然觀點 (Transcendental View)**：無法精確量化，但一體驗就能感受到其精緻、優雅與直覺的極致美感（如絲滑流暢的 UI/UX 與微互動）。
2. **使用者觀點 (User View - Fitness for Use)**：軟體是否能切中真實使用者的痛點、滿足業務需求並帶來實質效益（合用性）。
3. **製造觀點 (Manufacturing View - Conformance)**：軟體產出物與工程流程是否 100% 符合規格、通過靜態檢測與 Quality Gate（符合度）。
4. **產品觀點 (Product View - Architecture)**：產品內在結構特性，如高內聚低耦合、強固型態、可測試性與可維護性。
5. **價值觀點 (Value-based View - ROI)**：軟體帶來的商業價值與產出是否顯著高於其開發、測試與維運之總成本（投資報酬率）。

💡 五大品質觀點的深度工程實例與對照

- **1. 超自然觀點 (Transcendental View) —— 「說不出來哪裡好，但一用就被驚豔的靈魂美學」**
 - 例如：**Apple iOS** 介面的手勢滑動物理慣性阻尼、**Notion** 極簡直覺的斜線指令 (/)、或是頂級 IDE 行雲流水毫無卡頓的微互動。你無法用單一測試斷言定義它，但一旦觸碰，那種絲滑、精緻與直覺感會讓人發自內心讚嘆：「這就是極致的好品質！」
 - 反之：早期政府的報稅或學校選課系統，功能雖然全部齊全，但介面如同 90 年代的老舊表格，按鍵延遲、排版擁擠，操作時令人挫折焦躁。
- **2. 使用者觀點 (User View) —— 「合用才是王道 (Fitness for Use)」**
 - 例如：**Zoom** 在疫情期間擊敗眾多老牌視訊巨頭，並非因為底層技術最深奧，而是因為使用者「點一個連結 3 秒就能開會」，連長輩與學童都能無障礙上手，極致解決「我要立刻開會」的痛點。
 - 反之：工程團隊花了半年研發具備超高難度演算法、支援 50 種冷門格式的播放器，但終端使用者只想要一鍵播放 MP4。功能極其強大，但無人想用，淪為數位庫存 (Shelfware)。
- **3. 製造觀點 (Manufacturing View) —— 「分毫不差的規格符合度 (Conformance to Requirements)」**
 - 例如：**航太飛控系統或銀行核心帳務系統**。規格書明確定義「轉帳交易必須在 100ms 內完成，數值必須精確至小數點後 4 位」，實作程式碼就必須 100% 通過單元測試、靜態掃描與 Continuous Quality Gate，零偏差、零缺陷。
 - 盲點：若當初需求規格書本身就有嚴重的設計盲點（如急診室系統規格未考慮現場醫護動線），製造觀點就算拿下 100 分，也只是「分毫不差地造出了一套完全合規的廢品」。
- **4. 產品觀點 (Product View) —— 「內在架構與骨相之美 (Internal Characteristics)」**
 - 例如：**Linux 核心或 Spring Framework** 的架構設計。模組高度內聚、彼此鬆散耦合 (Loose Coupling)，圈複雜度極低，並具備 90% 以上自動化測試保護。即使歷經十餘年版本迭代，依然能穩健重構擴展。
 - 反之：**義大利麵程式碼 (Spaghetti Code)**。外表介面看起來堪用，但原始碼沒有分層、充斥複製貼上與全域變數，改動結帳頁面的一個按鈕顏色，竟導致會員登入功能全面崩潰。
- **5. 價值觀點 (Value-based View) —— 「商業性價比與投資報酬率 (ROI)」**

- 例如：新創團隊在驗證商業模式初期，採用 Serverless 與開源元件以極小預算在兩週內打造出 **MVP（最小可行產品）** 搶佔市場，以最低成本取得最大商業回饋。
- 反之：在產品商業模式尚未驗證前，工程團隊執意耗資數百萬引進複雜的分散式架構與自建機房，結果產品上線前資金便已耗盡，商業上宣告破產。

品質觀點	核心定義	軟體工程實例	忽略該觀點的後果
超自然觀點 (Transcendental)	無法精確量化，但一體驗就能感受到其精緻與美感	極致流暢的 UI/UX、細膩的動畫微互動 (iOS/Notion)	軟體感覺粗製濫造、冰冷卡頓，使用者體驗差
使用者觀點 (User View)	符合使用者真實需求與期望 (Fitness for Use)	解決使用者痛點、操作直覺易上手 (Zoom 一鍵入會)	功能很強但沒人想用，淪為陳列品 (Shelfware)
製造觀點 (Manufacturing View)	符合工程規格與標準流程 (Conformance)	遵循 Clean Code 規範、零規格偏離、通過 Quality Gate	規格本身有漏洞時，做出一套符合規格的垃圾
產品觀點 (Product View)	產品本身的內在技術特性與架構材質	高內聚低耦合、強固型態、低圈複雜度 (Linux/Spring)	架構腐化，改一個小功能引發全面崩潰
價值觀點 (Value-based View)	顧客願意支付的成本與性價比 (ROI)	軟體帶來的商業價值大於開發與維運成本 (精實 MVP)	開發成本失控超支，商業上不可行

- 👍 程式必須是為了給人看而寫，命令機器執行只是附帶任務。—— Abelson & Sussman
- 👍 品質不是動作，是一種習慣。—— Aristotle

🗣️ 文字雲互動：品質觀點

互動提問

你覺得哪一個觀點是最重要的品質指標？請寫下來。

課堂互動

🧠 概念核對問答 (CCQ 3)

問題

某專案團隊開發的電商 App 完全符合合約規格書上的每一條需求（製造觀點合格），但因為底層架構高度耦合且完全沒有寫單元測試，半年後客戶想新增一個促銷功能時，工程團隊發現必須重寫整個系統。這代表該軟體在 Garvin 的哪一個品質觀點上嚴重不及格？

- A) 產品觀點 (Product View)
 - B) 製造觀點 (Manufacturing View)
 - C) 法律合約觀點 (Legal Contract View) D) 超自然觀點 (Transcendental View)
-

課堂互動

1.4 軟體品質工程核心概念：V&V、品質成本 (CoQ) 與測試左移

1.4.1 驗證與確認 (Verification vs. Validation, V&V)

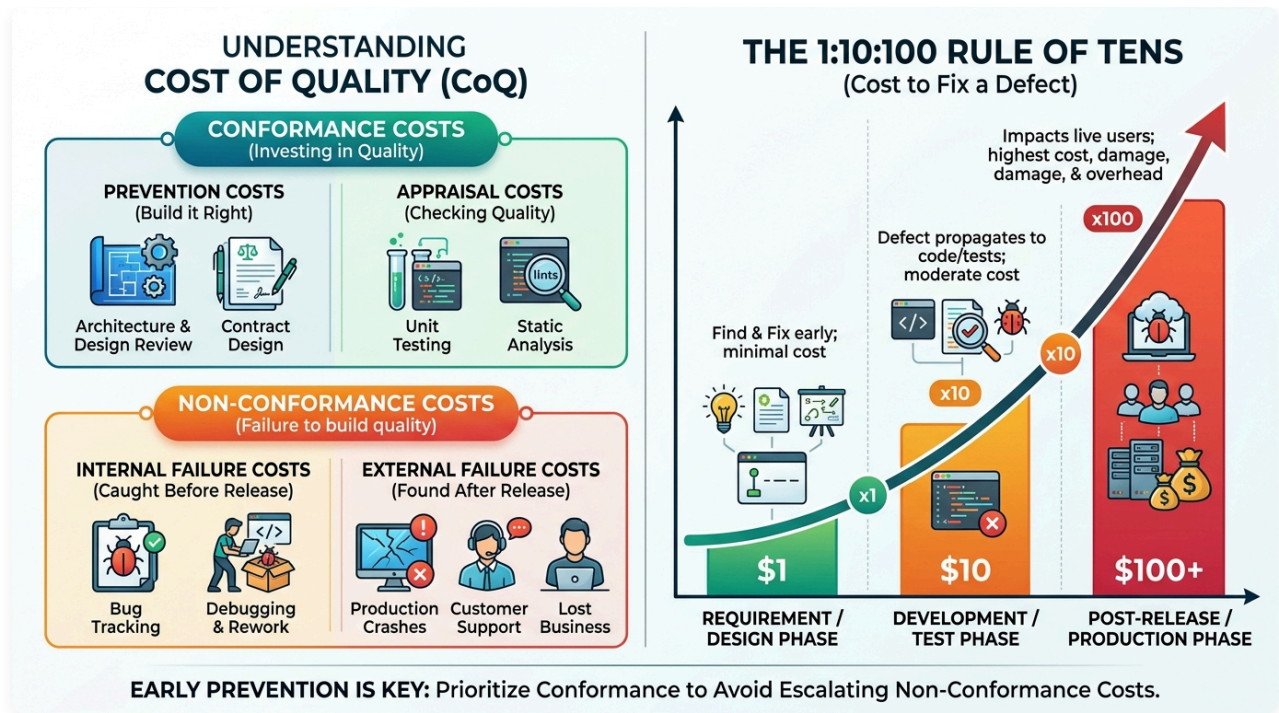
軟體測試與品質保證的靈魂大問：

-  **Verification (驗證)** : *Are we building the product **right**?* (我們是否有正確地建造軟體?)
-  **Validation (確認)** : *Are we building the **right** product?* (我們建造的是否是正確的軟體?)

- **Verification (驗證)**：確保軟體產出物符合上個階段設定的規格（檢視程式碼是否符合設計圖、單元測試是否符合規格）。
 - **Validation (確認)**：確保軟體真正滿足使用者的真實業務需求（驗收測試、易用性測試、現場 Beta 測試）。
-

1.4.2 軟體品質成本 (Cost of Quality, CoQ) 與 1:10:100 定律

軟體品質並不是「越完美越好」，而是在成本與效益之間取得最佳平衡。在軟體品質管理中，品質成本 (Cost of Quality, CoQ) 分為**一致性成本**與**非一致性成本**：



圖形解說：品質成本架構 (CoQ) 與 1:10:100 缺陷倍增定律

- **一致性成本 (Conformance Costs - 主動投資品質)：**
 - **預防成本 (Prevention)：** 架構審查、合約設計 (Design by Contract)、工程培訓與靜態程式碼規範。
 - **評估成本 (Appraisal)：** 單元測試 (Unit Tests)、靜態程式碼分析 (SonarQube) 與同行程式碼審查 (Code Review)。
- **非一致性成本 (Non-Conformance Costs - 忽視品質的慘痛代價)：**
 - **內部失敗成本 (Internal Failure)：** 上線前發現 Bug 導致的除錯 (Debugging)、重構與重測返工成本。
 - **外部失敗成本 (External Failure)：** 生產環境崩潰 (Outage)、客戶求償、緊急熱修復 (Hotfix) 與商譽破產。
- **1:10:100 定律 (The Rule of Tens)：**
 - 在需求/設計階段 發現並修復缺陷的代價為 \$1。
 - 若拖延至開發/測試階段 修復代價暴增至 \$10。
 - 若洩漏至產品發布後 (Production Phase)，修復代價與災難損失將高達 100 ~ 1000+ !
- **測試左移 (Shift-Left Testing)：** 將品質活動儘早移至生命週期前端，是降低軟體總擁有成本的最有效手段。

🧠 概念核對問答 (CCQ 4)

問題

某軟體團隊為醫院開發一套急診掛號分流系統。開發團隊嚴格按照原先簽訂的「系統需求規格書」完成所有功能實作，且單元測試與程式碼審查 (Code Review) 皆 100% 通過、完全無錯誤 (Bug)。但實際上線在急診室臨床試用時，醫護人員發現分流操作流程完全不符合急救現場的真實

節奏與急迫需求，導致無法在實務中使用。根據軟體工程定義，此系統在下列哪一項做得很好，但在哪一項嚴重失敗？

- A) Verification (驗證) 做得很好，但 Validation (確認) 嚴重失敗
- B) Validation (確認) 做得很好，但 Verification (驗證) 嚴重失敗
- C) Verification 與 Validation 兩者皆成功，純屬醫護人員操作習慣問題
- D) Verification 與 Validation 兩者皆失敗，因為使用者無法順利使用就代表底層邏輯有語法錯誤

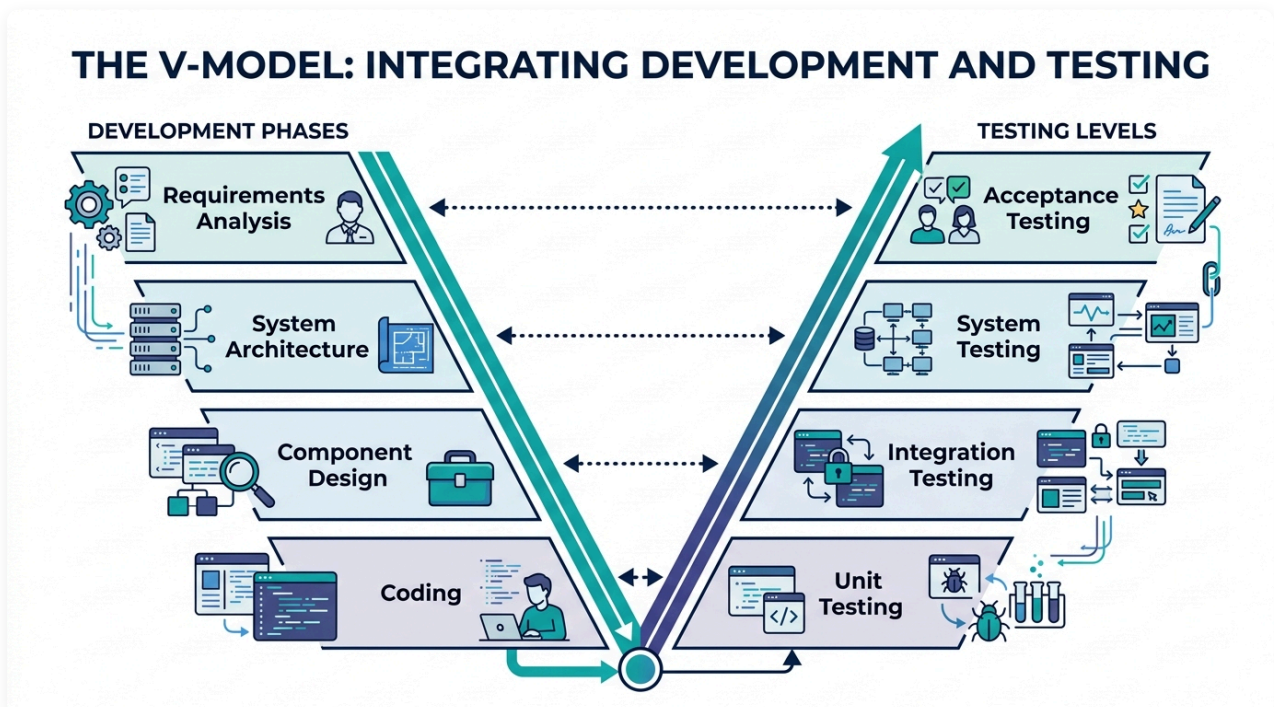
課堂互動

1.5 軟體工程流程與生命週期中的品質把關 (SDLC & CI/CD Quality Governance)

軟體工程的核心哲學在於：「品質不是最後靠測試敲打出來的，而是在整個生命週期中逐步建造並防護出來的 (Quality is built-in, not tested-in)。」

1.5.1 傳統模型與 V 模型：對稱性與早期測試規劃

在傳統線性模型（瀑布模型）中，測試常被延後至編程結束後才進行，落入 1:10:100 的高昂修復陷阱。為解決此問題，V 模型 (V-Model) 建立了開發階段與測試層級的嚴密對稱與平行規劃：



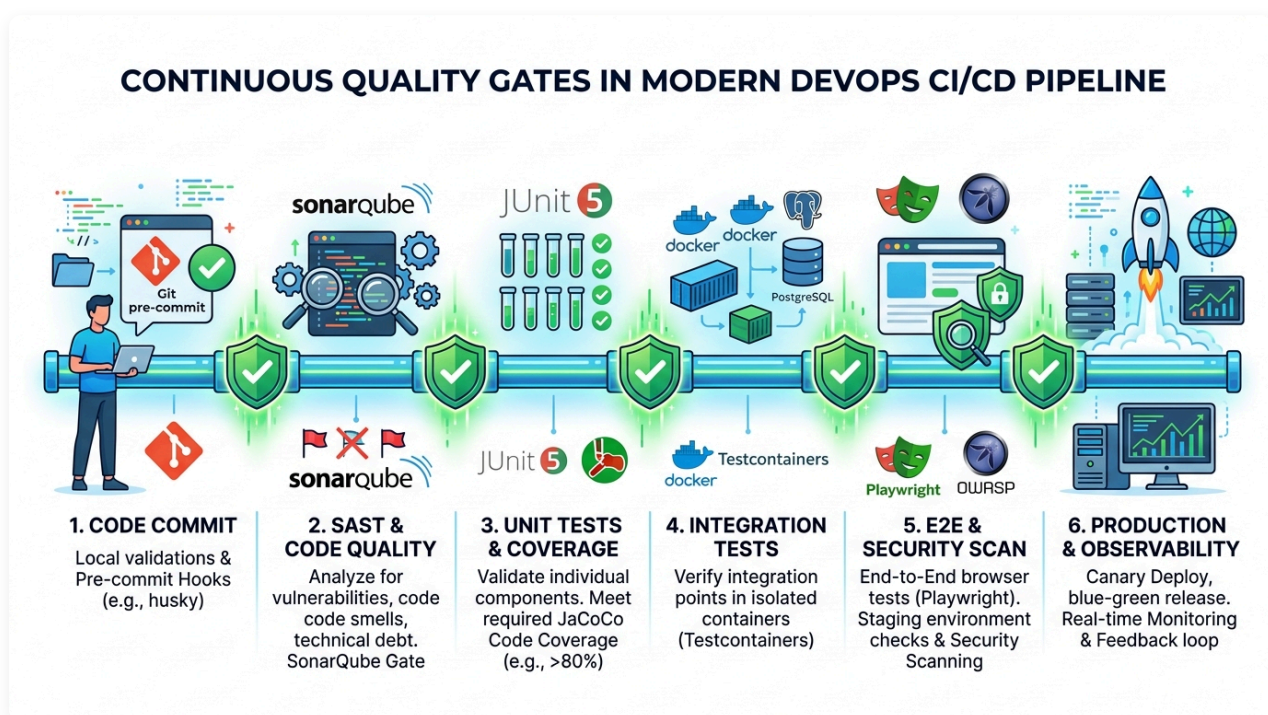
圖形解說：V 模型 (V-Model) 開發與測試層級對稱圖

- 左側：開發階段 (Verification) → 右側：測試層級 (Validation) 平行對稱：
 - i. 需求分析 (Requirements Analysis) → 同步規劃並設計 驗收測試 (Acceptance Testing)。

- ii. 系統架構 (System Architecture) → 同步規劃並設計 系統測試 (System Testing)。
- iii. 元件設計 (Component Design) → 同步規劃並設計 整合測試 (Integration Testing)。
- iv. 編寫程式碼 (Coding) → 實作並執行 單元測試 (Unit Testing)。
- 核心價值：在寫下第一行業務程式碼之前，驗收與整合測試的規格與邊界就已經隨同架構圖確立完成。

1.5.2 現代敏捷與 DevOps CI/CD 連續品質門檻 (Continuous Quality Gates)

在現代雲原生與微服務時代，軟體以每日甚至每小時的頻率持續交付。品質保證已全面升級為自動化流水線上的「連續品質門檻 (Continuous Quality Gates)」：



圖形解說：現代 DevOps CI/CD 流水線中的 6 大連續品質門檻

1. **Code Commit 門檻**：本地 Git Pre-commit Hook 自動執行程式碼格式化與快速靜態語法檢查。
2. **SAST 靜態程式碼品質門檻**：SonarQube / SpotBugs 掃描程式碼異味 (Code Smells)、技術債與 OWASP 安全弱點。
3. **Unit Tests & 覆蓋率門檻**：JUnit 5 執行毫秒級單元測試，並由 JaCoCo 驗證行覆蓋率與分支覆蓋率門檻（如 > 80%）。
4. **Integration Tests 容器整合門檻**：Testcontainers 一鍵拉起真實 Docker 容器（PostgreSQL / Redis），驗證真實資料庫存取與 API 契約。
5. **E2E & Security Scan 驗收門檻**：Playwright 自動化模擬真實使用者操作流程，搭配 OWASP ZAP 進行動態滲透掃描。
6. **Production & Observability 部署自愈門檻**：透過金絲雀 (Canary) 或藍綠部署平滑發布，並由可觀測性 (Observability) 系統即時監控 P99 延遲與異常告警。

👤 排序互動：V 模型（V-Model）開發與測試生命週期活動排序

問題

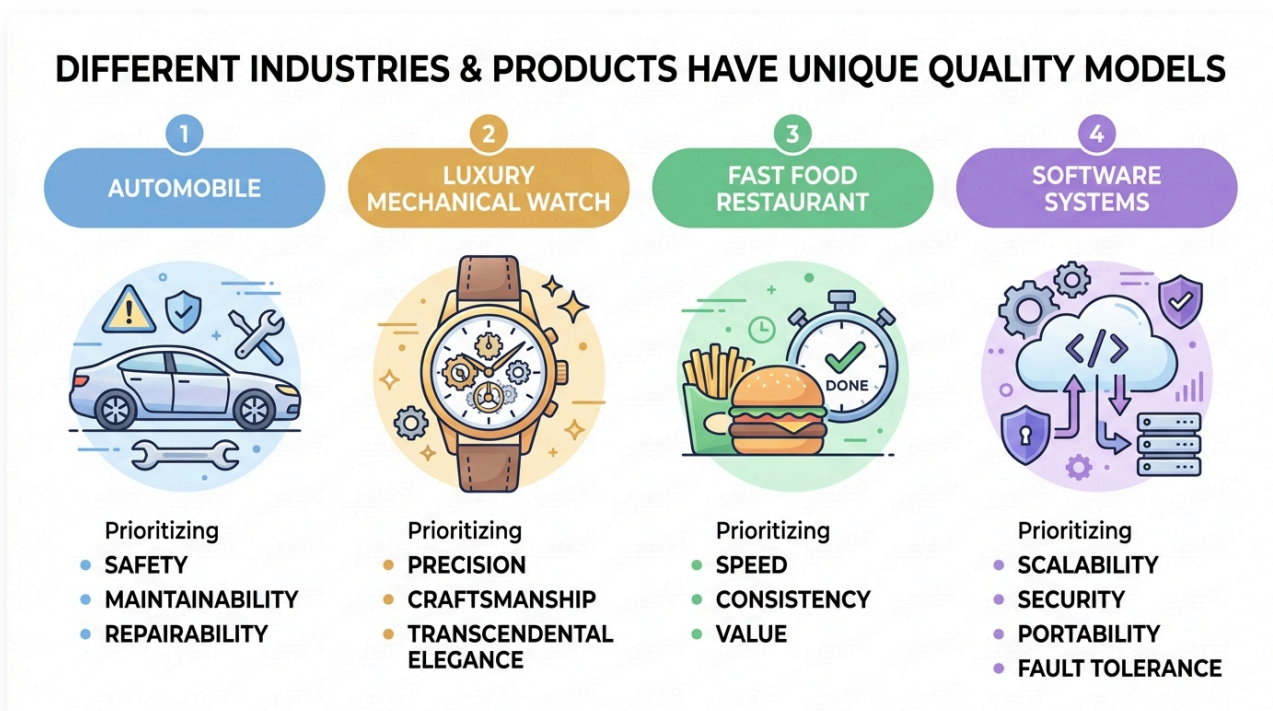
在傳統 V 模型（V-Model）中，軟體的「左側開發階段（規格制定與分解）」與「右側測試層級（組裝與驗證）」具有嚴密的對稱與依賴關係。請將下列 8 項軟體工程活動，依照**「實際執行生命週期順序（從最初需求分析到最終驗收）」**由先至後排列出正確順序：

1. 需求分析與規格定義 (Requirements Analysis)
2. 系統架構設計 (System Architecture Design)
3. 元件/模組詳細設計 (Component Design)
4. 程式碼編寫與實作 (Coding)
5. 單元測試執行 (Unit Testing)
6. 整合測試執行 (Integration Testing)
7. 系統測試執行 (System Testing)
8. 驗收測試執行 (Acceptance Testing)

課堂互動

1.6 現代軟體品質模型 (ISO 9126 → ISO 25010)

每一個產業都有其獨特的品質模型。例如製造簡易塑膠椅的廠商不會將「可維修性」列為核心品質指標，椅子壞了直接丟棄換新即可；但汽車產業就必須將「可維護性 (Maintainability)」與「安全性 (Safety)」置於最高優先級。



圖形解說：不同產業與物品具備截然不同的品質模型

1. 汽車產業 (Automobile)：優先著重於 安全性 (Safety)、可維護性 (Maintainability) 與 抗損耐用度 (Repairability)。
2. 精品機械錶 (Luxury Mechanical Watch)：優先著重於 走時精確度 (Precision)、工藝品質 (Craftsmanship) 與 超自然美感 (Transcendental Elegance)。
3. 速食餐廳 (Fast Food Restaurant)：優先著重於 出餐速度 (Speed)、口味一致性 (Consistency) 與 性價比 (Value)。
4. 軟體系統 (Software Systems)：優先著重於 高並發擴充性 (Scalability)、資訊安全 (Security)、跨平台移植性 (Portability) 與 容錯自癒力 (Fault Tolerance)。

1.6.1 ISO 25010 八大產品品質特性 (Product Quality Characteristics)

國際標準組織早期制定了 ISO 9126 (定義 6 大特性)；現代 ISO 25010 (SQuaRE, 軟體產品品質要求與評估標準) 進一步擴展為 8 大產品品質特性 (Product Quality)：



圖形解說：ISO 25010 八大產品品質特性（第一層核心維度）

1. 功能適合性 (Functional Suitability)

系統所提供的功能是否滿足明訂與隱含的業務需求：

- **功能完備性 (Completeness)**：功能涵蓋了所有指定的任務與使用者目標。
- **功能正確性 (Correctness / Accurateness)**：系統產出精確無誤的結果（例如：ATM 提款功能不僅能吐鈔，且扣款金額與吐鈔張數必須分毫不差）。
- **功能適切性 (Appropriateness / Suitability)**：功能是否符合軟體本質定位（例如：一款純文字 Markdown 筆記軟體若強行塞入線上聊天與直播功能，即違反適切性）。

2. 可靠性 (Reliability)

系統在特定條件與時限內維持指定效能水準的能力：

- **成熟度 (Maturity)**：在正常運作下避免發生故障的能力（低故障率）。
- **容錯度 (Fault Tolerance)**：當面對非法輸入、硬體異常或網路抖動時，系統依然能正常運作而不崩潰（例如：接收到畸形 JSON 時拋出友好提示而非直接當機）。
- **可回復性 (Recoverability)**：發生故障中斷後，重新建立服務並復原受影響資料的能力（例如：資料庫當機重啟後透過 WAL 日誌在秒級內回復資料一致性）。

3. 效能效率 (Performance Efficiency)

系統在特定條件下所展現的效能與資源消耗比例：

- **時間行為 (Time Behavior)**：系統處理請求的反應時間、延遲與吞吐量（例如：API P99 響應時間 < 200ms）。
- **資源利用率 (Resource Utilization)**：執行時所消耗的 CPU、記憶體、硬碟 I/O 與網路頻寬數量。
- **容量 (Capacity)**：系統能支援的最大並發連線數或資料儲存上限。

4. 易用性 (Usability)

使用者學習、操作與喜愛該系統容易程度：

- **易識別性 (Appropriateness Recognizability)**：使用者能否一眼看出該軟體是否符合其需求。
- **易學習性 (Learnability)**：新手使用者需要多少時間才能熟練掌握基本操作。
- **易操作性 (Operability)**：系統控制與操作是否直覺、流暢。
- **使用者錯誤防護 (User Error Protection)**：在使用者進行高危險操作前給予警告或確認（例如：格式化磁碟前跳出二次確認視窗）。

5. 安全性 (Security)

系統保護資訊與資料免受惡意攻擊與未授權存取的能力：

- **機密性 (Confidentiality)**：確保只有獲得授權的人員能存取敏感資料（如密碼加鹽雜湊存儲、傳輸全面 HTTPS 加密）。
- **完整性 (Integrity)**：防止未授權的修改或竄改。
- **抗抵賴性 (Non-repudiation)**：能證明特定動作或交易確實由特定人員發起（如數位簽章與不可篡改的稽核日誌）。
- **真實性 (Authenticity)** 與 **授權能力 (Accountability)**。

6. 可維護性 (Maintainability)

工程團隊修改、優化、修復或調適軟體的有效性與效率：

- **模組化 (Modularity)**：軟體由相對獨立的模組構成，修改單一模組不會對其他模組造成不可預期的連鎖破壞。
- **可分析性 (Analyzability)**：診斷軟體缺陷或評估修改影響的難易程度（例如：具備良好的結構化 Logging 與分散式追蹤 Tracing）。
- **可修改性 (Modifiability)**：在不降低整體品質的前提下修改程式碼的難易度。
- **可測試性 (Testability)**：為軟體建立測試並執行驗證的難易程度（高內聚低耦合的架構具備極高的可測試性）。

7. 可移植性 (Portability)

系統從一個硬體、軟體或作業環境轉移至另一環境的適應能力：

- **適應性 (Adaptability)**：在無需進行額外開發下適應不同作業系統或雲端平台的能力。
- **易安裝性 (Installability)**：在指定環境下成功部署並運行的簡易度。
- **易置換性 (Replaceability)**：在相同環境下替換同類軟體產品的能力（例如：使用 Docker 容器與 Testcontainers 實現開發機、CI 伺服器與生產環境的 100% 一致性）。

8. 相容性 (Compatibility)

系統與其他軟體產品在共享硬體或網路環境時的共處與互動能力：

- **共存性 (Co-existence)**：與其他獨立軟體共享公共資源（如記憶體、通訊埠）而互不干擾。
- **互通性 (Interoperability)**：透過標準協定或 API（如 REST / GraphQL / LDAP）與外部系統順暢交換資訊並協同作業。

1.6.2 ISO 25023：品質特性的量化指標與工程實務 (Quantitative Quality Metrics)

「如果你無法度量它，你就無法改善它。」 (If you cannot measure it, you cannot improve it. — 物理學家 凱爾文勳爵 / 軟體工程大師 Tom DeMarco)

ISO 25010 為軟體品質建立了完整且結構化的「**概念維度**」（8 大特性、31 個子特性）。然而在工程實踐中，單純要求「系統要有高可靠性」、「程式碼要容易維護」是無法驗證的空泛口號。

為了使品質要求可被精確評估與自動化把關，ISO/IEC 25000 (SQuaRE) 系列進一步制定了 **ISO/IEC 25023 (Measurement of system and software product quality)**，為 ISO 25010 的各項品質特性定義了具體的**量化測量函數與計算公式 (Measurement Functions)**。在現代 SQA 與 DevOps 體系中，這些量化指標轉化為系統的 **SLA / SLO（服務水準協議與目標）**、**CI/CD 自動化測試門檻** 與 **SonarQube Quality Gates（品質閘門）**。

ISO 25010 八大特性量化指標全覽表

品質特性	核心子特性	核心量化指標 (Metrics / KPIs)	工業標準 / 建議門檻	常見工程量測工具
1. 功能適合性 (Functional Suitability)	完備性 正確性 適切性	• 需求覆蓋率： $\frac{\text{已實作驗證需求數}}{\text{總規格需求數}} \times 100$ • 驗收測試通過率： $\frac{\text{通過案例數}}{\text{總測試案例數}} \times 100$ • 功能缺陷密度 (Defect Density)：每 KLOC 或每功能點的功能 Bug 數	• 需求覆蓋率 = 100 • 驗收通過率 ≥ 99.5 • 上線前 Critical Bug = 0	Jira, TestRail, JUnit, Allure
2. 可靠性 (Reliability)	可用性 容錯度 成熟度 可回復性	• 系統可用度 (Availability)： $\frac{\text{總運作時間} - \text{故障停機時間}}{\text{總運作時間}} \times 100$ • 平均故障間隔時間 (MTBF)：正常運行的平均時數 • 平均修復時間 (MTTR)：從故障到恢復運行的平均時間 • 容錯回復率：異常注入時成功降級/自癒的比例	• 可用度達 99.9% (三條九) 或 99.99% (四條九) • MTTR < 15 分鐘 • 異常輸入當機率 = 0	Chaos Mesh, Datadog, Sentry, Prometheus
3. 效能效率 (Performance Efficiency)	時間特性 資源利用容量	• 回應時間 (Latency)：平均延遲、P95 / P99 延遲 • 吞吐量 (Throughput)：每秒交易/請求數 (TPS / QPS) • 資源利用率：尖峰 CPU / 記憶體 / 網路 I/O 佔用率 • 最大並發量：滿足延遲 SLA 下的最大同時連線數	• API P99 回應時間 < 200ms • 尖峰 CPU < 70、記憶體無洩漏 • 支援預期尖峰 1.5× 並發量	JMeter, k6, Locust, Grafana
4. 易用性 (Usability)	易學性 可	• 任務完成率 (Task Completion Rate)：首次順利完成核心操作比率 • 任務完成耗時 (Time-on-	• 核心任務完成率 ≥ 90 • SUS 分數	Google Lighthouse, Hotjar, UserTesting, Axe

品質特性	核心子特性	核心量化指標 (Metrics / KPIs)	工業標準 / 建議門檻	常見工程量測工具
	操作性 錯誤保護 可存取性	Task) ：新手與熟練者平均花費時間 • SUS 易用性評分 (System Usability Scale) ：0~100 分標準問卷 • 無障礙合規度 ：符合 WCAG 2.1 AA 規範檢查	≥ 68 ~ 70 分 • 破壞性操作 100 具備二次確認與復原機制	
5. 安全性 (Security)	機密性 完整性 真實性 抗抵賴性	• 已知弱點數 (CVE Count) ：重大/高危害漏洞數量 • CVSS 弱點嚴重程度分級 ：相依套件與原始程式碼安全評分 • 敏感資料加密合規率 ：靜態儲存與傳輸加密比例 • 安全弱點修復天數 (Security MTTR) ：發現到修補上線時間	• Critical / High 漏洞數 = 0 • 100% 採用 TLS 1.3 / AES-256 加密 • 高危害漏洞在 24 小時內修補完畢	OWASP ZAP, Snyk, SonarQube, Trivy
6. 可維護性 (Maintainability)	模組化 可分析性 可修改性 可測	• 圈複雜度 (Cyclomatic Complexity, CC) ：單一函式獨立路徑數 • 程式碼涵蓋率 (Code Coverage) ：單元測試行/分支涵蓋率 • 重複程式碼比例 (Duplication %) ：重複出現的行數百分比 • 可維護性指數 (Maintainability Index, MI) ：綜合架構評分 (0~100)	• 單一函式 $CC \leq 10$ • 行涵蓋率 ≥ 80、分支涵蓋率 ≥ 70 • 重複程式碼率 < 3 • MI 指數 ≥ 65 ~ 85 分	SonarQube, JaCoCo, PMD, ESLint

品質特性	核心子特性	核心量化指標 (Metrics / KPIs)	工業標準 / 建議門檻	常見工程量測工具
	試性			
7. 可移植性 (Portability)	適應性 易安裝性 易置換性	• 自動化部署成功率：無人值守 CI/CD Pipeline 部署通過率 • 環境啟動時間 (Boot Time)：容器映像檔建置與冷啟動秒數 • 平台移植工作量比率 (Porting Effort)：移植新平台工時 / 原開發工時	• 自動化部署成功率 ≥ 99 • 容器冷啟動時間 < 5 秒 • 移植工作量比率 ≤ 10	Docker, Kubernetes, Testcontainers, Terraform
8. 相容性 (Compatibility)	共存性 互通性	• 跨平台/瀏覽器相容測試通過率：目標支援矩陣測試通過率 • API 契約合規率 (Contract Compliance)：OpenAPI 規格相符比例 • 資源共存衝突率：因通訊埠/共用記憶體搶佔引發的異常次數	• 目標矩陣（主流瀏覽器/OS）相容率 = 100 • API 契約測試通過率 = 100 • 資源衝突失敗數 = 0	Playwright, Selenium Grid, Pact, Postman

現代 SQA 量化落地的「三大工程支柱」

1. 靜態程式碼檢測 (Static Quality Gates)：

- 在開發者提交程式碼時，由 CI 流程觸發 **SonarQube / PMD**，自動卡關圈複雜度、重複程式碼、單元測試涵蓋率與安全弱點，未達標則嚴格禁止合併 (Merge)。

2. 動態負載與效能測試 (Dynamic Performance Gates)：

- 在發布至預發 (Staging) 環境時，透過 **JMeter / k6** 執行壓測，確保 API 回應時間 (P95/P99) 與吞吐量符合 SLA 指標，防止效能退化 (Performance Regression)。

3. 運行時可靠性與監控 (Runtime Observability & SLI/SLO)：

- 系統上線後，利用 **Prometheus / Grafana / Datadog** 監控系統可用度 (如 99.99%)、平均修復時間 (MTTR) 與系統錯誤率，達到品質問題的主動告警與自癒。

課堂挑戰遊戲：ISO 25010 八大產品品質特性情境連連看 (10 題連環戰)

遊戲規則：

以下列出 10 個軟體工程日常開發、維運或慘痛故障的真實議題與事件。請根據 ISO 25010 八大產品品質特性，判斷每一項情境最主要是在考驗或違反哪一項品質特性？

【八大品質特性選項池】：

A. 功能適合性 (Functional Suitability) | B. 可靠性 (Reliability) | C. 效能效率 (Performance Efficiency) | D. 易用性 (Usability)
E. 安全性 (Security) | F. 可維護性 (Maintainability) | G. 可移植性 (Portability)
| H. 相容性 (Compatibility)

- **第 1 題【吐鈔卡死危機】**：使用者在 ATM 提款 10,000 元，系統扣款成功並列印明細，但吐鈔口機械卡死分文未出，帳戶卻已被扣款。
- **第 2 題【雙十一流量海嘯】**：電商平台午夜開賣，瞬間湧入 50 萬人搶購，伺服器 CPU 飆到 100%，API 響應時間從 150ms 暴增至 40 秒，大量連線超時。
- **第 3 題【致命的相鄰按鈕】**：雲端後台介面將「重啟伺服器」與「永久銷毀主機」按鈕放在相鄰位置且顏色相同，點擊時缺乏防呆二次確認，導致維運工程師手滑刪除正式環境資料庫。
- **第 4 題【牽一髮動全身的義大利麵】**：工程團隊想在會員資料中新增一個「暱稱」欄位，結果引發購物車、金流與推薦引擎等 8 個模組連鎖編譯錯誤，耗費 3 天重構修復。
- **第 5 題【斷電重啟秒級自癒】**：微服務資料庫節點突發斷電，備援機制在 3 秒內自動完成容錯移轉 (Failover)，並重放 WAL 日誌確保交易零遺失，外部連線僅感知微小抖動。
- **第 6 題【跨系統托運單格式打架】**：電商系統與黑貓宅急便 API 進行跨系統資料交換，因雙方日期協定格式不符 (YYYY-MM-DD vs DD/MM/YYYY)，造成所有物流單批次傳送失敗。
- **第 7 題【URL 改個數字看光他人隱私】**：駭客在瀏覽器將個人資料頁的 URL 從 `userId=1001` 改為 `userId=1002`，系統竟然毫無攔截，直接秀出另一位顧客的信用卡卡號與地址。
- **第 8 題【Mac 開發很順，推上 Linux 容器全掛】**：開發者在 macOS 本地端測試正常的服務，部署至生產環境的 Linux Docker 容器時，因寫死路徑大小寫 (Linux 嚴格區分大小寫) 導致找不到檔案崩潰。
- **第 9 題【地下室離線暫存與自動重送】**：外送員騎車進入收訊不良的地下停車場，手機 App 自動切換為離線模式快取送達狀態，當回到地面偵測到 5G 訊號時自動重送同步。
- **第 10 題【容器映像檔一鍵秒級部署】**：後端微服務封裝成標準 Docker 映像檔，無論部署在 AWS ECS、GCP GKE 還是地端 Kubernetes，皆能在 10 秒內透過統一設定檔一鍵拉起成功運行。

課堂互動

1.7 綜合練習與思維激盪

1. AI 時代的品質反思：

- 當生成式 AI 可以在幾秒鐘內產生包含完整 Javadoc 的程式碼時，為什麼軟體測試工程師的價值反而大幅提升？請從「Test Oracle 問題」與「自我印證偏誤」兩方面進行說明。

2. ISO 25010 維度分析：

- 請分析「微服務系統在後端資料庫當機重啟後，能在 5 秒內自動重新連線並將失敗的訊息重試成功，完全不丟失任何交易」，這體現了 ISO 25010 中的哪些品質特性？

3. 愛國者飛彈與數值精度：

- 試寫一小段 Java 程式碼，連續將 0.1 累加 1,000,000 次，比較其結果與 100000.0 的差異。觀察浮點數在長時間累計下的偏差現象。

附錄：課堂互動與概念檢核參考解答

為避免在課堂閱讀時影響同學的獨立思考，本章所有概念核對 (CCQ)、排序互動與遊戲挑戰之完整答案與深度解析統一收錄於此：

- ▶  點擊展開：CCQ 1 答案與解析（愛國者反導彈事件）
- ▶  點擊展開：CCQ 2 答案與解析（程式碼流失率 Code Churn）
- ▶  點擊展開：CCQ 3 答案與解析（Garvin 品質觀點）
- ▶  點擊展開：CCQ 4 答案與解析（Verification vs. Validation）
- ▶  點擊展開：排序互動答案與解析（V 模型活動排序）
- ▶  點擊展開：ISO 25010 八大品質特性情境連連看（10 題連環戰）答案與解析